# Musterlösung: Netzwerkanalyse mit praktischen Tools

## Aufgabe 1: `traceroute`

Beispielausgabe:

1 fritz.box (192.168.0.1) 2 100.64.0.1 3 ... 9 fra15s36-in-f4.1e100.net (142.250.186.132)

**Lösungen:**

a) Etwa 9 Hops bis zu Google.

b) * steht für einen Knoten, der nicht antwortet (Timeout oder blockiert ICMP).

c)

d) Wortspiel: 1*10^100 wird als ein Gogol bezeichnet.

e) traceroute sendet mehrere requests, wobei mit TTL=1 begonnen wird - dann antwortet der erste Host/Router, dann TTL=2, ..., bis irgendwann der angefragte Host antwortet.

## Aufgabe 2: `nslookup`

nslookup www.google.com

Beispielausgabe:

Server: dns.google Address: 8.8.8.8

Name: www.google.com Addresses: 142.250.186.132

Lösungen:

```
IP-Adresse: z. B. 142.250.186.132

DNS-Server: z. B. 8.8.8.8 (Google DNS)

Bei nicht existierendem Namen: „Non-existent domain“ oder Fehler
```

## Aufgabe 3: HTTP-Header im Browser

Beispielhafte Beobachtungen (Wikipedia):

```
Statuscode: 200 OK

HTML-Größe: ca. 5–20 kB

Größte Datei: ein Bild oder JavaScript-Datei

Bei Eingabe von wikipedia.org wird ein Redirect auf https://www.wikipedia.org/
ausgeführt → Status 301 oder 302
```

---

## Aufgabe 4: ping

```
ping www.google.com

Beispielausgabe:

64 bytes from fra15s36-in-f4.1e100.net: icmp_seq=1 ttl=117 time=17.4 ms

Lösungen:

    Durchschnittliche Antwortzeit: ca. 10–40 ms (abhängig vom Standort)

    Gesendete/empfangene Pakete: 4/4

    Keine Paketverluste → gute Verbindung
```

---

## Aufgabe 5: ARP

```
Eigene Adresse:

ip a

Beispiel:

IP: 192.168.0.100
MAC: 00:1a:2b:3c:4d:5e

Nach ping 192.168.0.1 und arp -a:

192.168.0.1    00:11:22:33:44:55

Lösungen:

    ARP zeigt die MAC-Adresse des Routers.

    Nach Anpingen eines neuen Hosts taucht dessen IP/MAC im Cache auf.
```

```
    Nach Leeren des Caches (z. B. arp -d) wird die Zuordnung neu gelernt.
```

## Aufgabe 7

a) Berechnen Sie die CRC Prüfsumme für die Daten D=111101 und das Generator-Polynom G=1011.

Lösung:

Die Länge von G ist 4, also müssen wir 3 Nullen an D anhängen und dann mit XOR dividieren:

```
111101000 : 1011
1011
----
 1000
 1011
 ----
   1110
   1011
   ----
    1010
    1011
    ----
       10
```

Die Prüfsumme muss ein bit kürzer sein als das Polynom, also hier `010`.

b) Beim Empfänger kommt ein Frame mit folgendem Inhalt an: 1011010110 an (gleiches Generator-Polynom). Wurden die Daten korrekt übermittelt?

Lösung:

```
1011010110 : 1011
1011
----
    0101
    1011
    ----
    1110
    1011
    ----
     1011
     1011
     ----
         0
```

Die Prüfsumme passt also zu den Daten, da der Rest der Division `0` ist.